

UC LINA 600 LINEAR ALGEBRA FOR GRAPHICS · MODULE 12 · LECTURE NOTES

Least Squares

What to do when there is no exact answer.

MODULE OUTCOMES

Explain why overdetermined systems have no solution.

Derive the normal equations from projection.

Fit a line and a plane to data.

Explain conditioning and why QR is preferred.

Recognise least squares in the program ahead.

1 The situation

WITH MORE EQUATIONS THAN UNKNOWNNS THERE IS USUALLY NO EXACT SOLUTION, SO YOU SETTLE FOR THE CLOSEST THING

Fit a straight line to twenty noisy measured points: **two unknowns (slope and intercept) and twenty equations** — **and no line passes through all twenty points**, so the system $Ax = b$ simply has no solution.

Which means, geometrically, that b is not in the column space of A (Module 02 §2's span) — it lies outside the set of vectors A is capable of producing — and that is the precise statement of the difficulty.

So instead find the x that makes Ax as close to b as possible — minimising the length of the residual $b - Ax$ — which is what the name 'least squares' refers to, since minimising a length means minimising a sum of squares.

And 'closest' means exactly the projection of b onto the column space (Module 05 §2) — which is why that module comes before this one, and makes the whole derivation three lines (§2).

2 The normal equations

WE WANT Ax as close to b as possible.
 The closest point in the column space is the
 PROJECTION of b onto it -- and the residual
 $(b - Ax)$ must therefore be perpendicular to that
 space (Module 05 section 2: the same condition,
 applied to a bigger object).

PERPENDICULAR TO THE COLUMN SPACE means
 perpendicular to every column of A , that is

$$\begin{aligned} A^T (b - Ax) &= 0 \\ A^T b - A^T A x &= 0 \\ A^T A x &= A^T b \quad \text{<-- the normal equations} \end{aligned}$$

AND THAT IS THE DERIVATION. Solve that square
 system and you have the least squares fit.

FOR A LINE FIT $y = mx + c$, A has columns $[x, 1]$, and
 the normal equations reduce to the familiar
 slope-and-intercept formulas -- which are worth
 deriving once rather than memorising.

The residual is perpendicular to the column space — the same condition as projecting onto a single vector, with a bigger target — which is why Module 05 §2's projection and this are fundamentally the same derivation. If you can derive one you can derive the other, and neither needs memorising.

3 Conditioning

FORMING $A^T A$ SQUARES THE CONDITION NUMBER, WHICH CAN DESTROY HALF YOUR PRECISION

The normal equations are mathematically correct and are a poor numerical algorithm — computing $A^T A$ roughly squares the condition number of the problem, so a mildly awkward system becomes a badly conditioned one before you have solved anything.

Which in double precision can cost you around eight significant digits — on a problem that was perfectly solvable before you formed the product — and the loss is silent.

So real implementations use QR decomposition (Gram-Schmidt's industrial relative, Module 05 §4) — which solves the identical problem without ever forming $A^T A$ — or the singular value decomposition when the matrix may be rank-deficient and you need the minimum-norm solution.

And this is the general pattern of numerical work: the derivation that explains the answer and the algorithm that computes it are frequently different things — which is Module 10 §2's characteristic polynomial point arriving a second time, and is worth generalising from.

4 Where it appears

- CSCE 633: linear regression is least squares, and the normal equations are the closed-form solution that every machine learning course opens with before moving to iterative methods for larger problems.
- CSCE 753: camera calibration, homography estimation, and bundle adjustment — all overdetermined systems built from noisy measurements, and all solved by least squares or a robust variant of it.
- CSCE 645: surface fitting and mesh parameterisation — minimising a quadratic energy over a mesh is a least squares problem, usually a sparse one, and the Laplacian systems of that course are exactly this.
- CSCE 649: constraint solvers, where more constraints than degrees of freedom is the normal situation rather than an exception, and some constraints must be satisfied only approximately.
- And anywhere at all that you fit something to measurements — which is what makes this the most broadly applicable module in the course. Linear regression is least squares, so this module is a machine learning prerequisite as much as a graphics one.

AND WHAT IT ASSUMES, WHICH IS WORTH STATING

Least squares minimises the sum of squared errors, which means **a single large outlier is weighted as heavily as many small errors combined** — so one bad measurement can dominate the entire fit and pull the line visibly away from the data.

Which is a modelling assumption rather than a mathematical necessity — **squared error is the right choice under independent Gaussian noise** (it is the maximum likelihood estimate there) **and the wrong choice under heavy-tailed noise or gross outliers**.

And robust alternatives exist — **minimising absolute error, Huber loss, or RANSAC** — which CSCE 753 uses constantly, because image feature correspondences contain not just noise but outright wrong matches.

So the honest claim is: 'the least squares fit, which assumes the errors are independent and approximately Gaussian' — which is this program's closing rule appearing in a prerequisite, and is a good sign that the rule is not decoration.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
MIT 18.06, the least squares lectures (free video)	§§1 and 2 — projection and the normal equations, with the column-space picture that makes it obvious.
3Blue1Brown, the projection material (free)	§1 — the geometric picture of projecting onto a subspace.
Trefethen & Bau, lectures 11 and 18 to 20	§3 — why QR rather than the normal equations, with the conditioning analysis. Library copy.
Hastie, Tibshirani & Friedman — Elements of Statistical Learning (free PDF)	§4's first row — regression as least squares, free, and the direct bridge to CSCE 633.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Set up an overdetermined system** and confirm it has no solution.
- 2 **Show that $\mathbf{b}$ is outside the column space**, explicitly.
- 3 **Derive the normal equations** from perpendicularity.
- 4 **Fit a line to twenty noisy points**, by hand and by code.

- 5 **Derive the slope and intercept formulas** from the normal equations.
- 6 **Fit a plane to three-dimensional points.**
- 7 **Construct an ill-conditioned problem** and compare normal equations to QR.
- 8 **Measure the digits lost** in each.
- 9 **Add one gross outlier** and watch the fit move.
- 10 **Implement a robust alternative** and compare on the same data.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Why does an overdetermined system usually have no solution?
- 2 State that geometrically, in terms of the column space.
- 3 What does 'closest' mean here?
- 4 Derive the normal equations.
- 5 Why is this the same derivation as projecting onto a vector?
- 6 What is wrong with the normal equations numerically?
- 7 What do real implementations use instead?
- 8 State the general pattern this illustrates.
- 9 Name four places least squares appears in the program.
- 10 What does least squares assume, and what are the alternatives?